Resumen del proyecto

Este proyecto fue desarrollado como una aplicación híbrida utilizando lenguaje C y ensamblador NASM para arquitectura x86-64. El programa principal fue implementado en C, mientras que las funciones principales solicitadas fueron desarrolladas en archivos separados de ensamblador: swap.asm, sum.asm y bubble_sort.asm.

En el archivo main.c se implementó la interacción con el usuario. El programa solicita por pantalla el tamaño del arreglo, validando que el valor ingresado esté dentro del límite permitido. Para evitar el uso de memoria dinámica, se declaró un arreglo de tamaño fijo mediante la constante MAX_SIZE. Si el usuario ingresa un tamaño menor que 1 o mayor que el límite permitido, el programa muestra un mensaje de error y vuelve a solicitar el dato.

Después de validar el tamaño, el programa pide al usuario ingresar los valores del arreglo. Luego llama a la función sum, implementada en ensamblador, para calcular la suma total de los elementos. Posteriormente llama a la función bubble_sort, también implementada en ensamblador, para ordenar el arreglo de forma ascendente. Finalmente, el programa muestra en pantalla el arreglo ordenado y la suma total.

También se prueba la función swap, la cual recibe las direcciones de dos variables enteras y realiza el intercambio de sus valores. En C, esta función se llama pasando las direcciones de las variables mediante el operador &, por ejemplo: swap(&x, &y).

Detalles importantes de la implementación en ensamblador

El proyecto utiliza la convención de llamadas de Windows x64. Según esta convención, los primeros parámetros de una función se reciben en registros específicos:

- RCX recibe el primer parámetro.
- RDX recibe el segundo parámetro.
- R8 y R9 se usan para parámetros adicionales.
- EAX o RAX se utiliza para retornar valores.

En la función swap, RCX contiene la dirección de la primera variable y RDX contiene la dirección de la segunda variable. Para realizar el intercambio, se utiliza EAX como registro temporal para guardar el valor de la primera variable y R8D para guardar el valor de la segunda variable. Luego se escriben los valores intercambiados en las direcciones originales.

En la función sum, RCX contiene la dirección base del arreglo y EDX contiene el tamaño del arreglo. El registro EAX se utiliza como acumulador de la suma, mientras que R8D funciona como índice del ciclo. Como cada entero ocupa 4 bytes, el acceso a cada elemento del arreglo se realiza mediante una dirección calculada de la forma [RCX + R8 * 4].

En la función bubble_sort, RCX contiene la dirección del arreglo y EDX el tamaño. Se utilizan varios registros para controlar los ciclos del algoritmo: R8D representa el índice externo, R9D representa el índice interno, R10D se usa como límite de comparación en cada pasada, EAX guarda temporalmente el valor de arr[j] y

R11D guarda el valor de arr[j + 1]. Si el valor actual es mayor que el siguiente, se intercambian directamente en memoria.

El ensamblador NASM genera archivos objeto .obj para cada archivo .asm. Luego Visual Studio enlaza esos archivos objeto con el archivo main.c para producir el ejecutable final del proyecto. De esta forma, el programa en C puede llamar correctamente a las funciones implementadas en ensamblador.